

第 15 章 AI Agent 岗位简历模板

本章定位：提供经过优化的简历模板、项目经历描述范例和简历撰写技巧，帮助求职者打造高通过率的 AI Agent 方向简历。

目录

- 一、简历撰写核心原则
- 二、技能关键词列表
- 三、Nanobot 项目简历模板
- 四、智能运维助手项目简历模板
- 五、多平台客服 Bot 项目简历模板
- 六、自定义 MCP Server 开发项目简历模板
- 七、其他可写项目方向
- 八、自我评价模板
- 九、完整简历示例
- 十、简历注意事项
- 十一、不同岗位的简历侧重点
- 十二、简历投递策略

一、简历撰写核心原则

1.1 STAR 量化原则

每段项目经历都要遵循 STAR 法则并尽量量化成果：

- **S (Situation)**：项目背景——什么场景、什么规模
- **T (Task)**：你的任务——负责什么、目标是什么
- **A (Action)**：你的行动——用了什么技术、做了什么决策
- **R (Result)**：成果——提升了多少、解决了什么问题

反面示例（空洞无力）：> 负责项目开发工作，参与了相关功能的设计和实现。

正面示例（具体有力）：> 主导设计并实现了基于 ReAct 循环的 AI Agent 系统，集成 12 个 MCP 工具，支撑日均 5000+ 次对话请求，用户问题解决率从 60% 提升至 85%。

1.2 六大黄金法则

1. 数字说话：尽量用数字量化每一条成果（提升 XX%、减少 XX 毫秒、覆盖 XX 个场景）
2. 技术栈明确：在每个项目中明确列出使用的技术栈
3. 深度优于广度：2 个有深度的项目 > 5 个浅尝辄止的项目
4. 关键词匹配：根据目标 JD 调整关键词的顺序和权重
5. 「做了什么」不如「解决了什么问题」：结果导向，不是任务清单
6. 一页为佳：除非有 10 年 + 经验，简历控制在一页 A4 纸内

1.3 AI Agent 方向简历的特殊性

AI Agent 方向的简历与传统后端开发简历的不同之处：

- 源码理解加分：面试官特别看重候选人是否读过 Agent 框架的源码，而非只是调用 API
- 协议理解加分：对 MCP 等协议的理解比单纯会用某个框架更有含金量
- 端到端经验：从 Prompt 设计到系统部署的全链条经验比单点能力更有竞争力
- 开源贡献：有 GitHub 贡献或技术博客会显著提升简历吸引力

二、技能关键词列表

按权重排序（从高到低）

第一梯队（必须包含）

Python, AI Agent, LLM/大语言模型, Prompt Engineering, RAG, Tool Calling/Function Calling, 异步编程

第二梯队（强烈推荐）

MCP 协议, Agent 框架(LangChain/Nanobot), 向量数据库, Transformer 架构, FastAPI, Docker, Git

第三梯队（加分项）

多Agent编排, SFT/RLHF/DPO, LoRA, PyTorch, 系统设计, CI/CD, Kubernetes, Redis, PostgreSQL

第四梯队（差异化项）

MCP Server 开发, Agent 源码研究, Agent 系统设计, 开源贡献, 技术博客, Prompt Cache 优化

技能展示模板

技术技能:

- 编程语言: Python (精通)、TypeScript (熟悉)、Go (了解)
 - AI/LLM: Prompt Engineering、RAG 系统设计、Agent 架构设计、MCP 协议
 - Agent 框架: Nanobot (源码级理解)、LangChain/LangGraph (项目实践)
 - 模型训练: PyTorch、Transformer 架构、SFT/DPO、LoRA 微调
 - 数据库: PostgreSQL、Redis、Milvus (向量数据库)
 - 工程工具: Docker、Git、CI/CD、FastAPI、asyncio
-

三、Nanobot 项目简历模板

模板 A: Agent 框架源码研究项目（适合偏理论/架构方向）

项目名称: 基于 HKUDS/Nanobot 的 AI Agent 系统源码研究与二次开发

项目周期: 2026.02 - 2026.04

项目角色: 独立研究 / 核心开发

技术栈: Python 3.12, HKUDS/nanobot, MCP 协议, Anthropic Claude API, asyncio, JSON-RPC 2.0

项目描述:

深入研究开源 AI Agent 框架 Nanobot (GitHub 37K+ stars) 的完整源码, 理解其 Agent 循环、记忆管理、工具注册、MCP 集成等核心机制, 并基于此进行二次开发和能力扩展。

核心职责:

- 系统研读 Nanobot 核心源码 (约 4000 行 Python), 完成 8 个核心模块 (AgentLoop、ContextBuilder、MemoryConsolidator、ToolRegistry、SubagentManager、MessageBus、MCPClient、ChannelAdapter) 的架构分析和流程图绘制
- 深入分析 AgentLoop 的 ReAct 执行流程, 理解 Thought-Action-Observation 循环的工程实现, 梳理从用户输入到最终回复的完整数据流
- 研究 MemoryConsolidator 的双层记忆架构 (会话内压缩 + 跨会话持久化), 理解 save_memory 虚拟工具的设计哲学

- 分析并发控制设计：session_locks（会话级互斥）和 _concurrency_gate(3)（全局信号量）的分层并发策略

项目成果：

- 输出 12 篇深度技术文档（共 5 万+ 字），覆盖架构、源码、协议等维度
 - 基于源码理解开发 3 个自定义 MCP Server（MySQL 查询、文档检索、日程管理）
 - 成功将 Nanobot 集成到飞书机器人，实现团队内部 AI 助手功能
 - 形成一套可复用的 Agent 框架评估方法论，从 7 个维度对比 Nanobot 与 LangChain
-

模板 B：AI 助手应用项目（适合偏工程/产品方向）

项目名称：基于 Nanobot 的企业智能助手系统

项目周期：2026.02 - 2026.04

项目角色：后端开发 / 技术负责人

技术栈：Python, HKUDS/nanobot, MCP 协议, Claude API, FastAPI, PostgreSQL, Redis, Docker, 飞书开放平台

项目描述：

基于 Nanobot Agent 框架构建企业级 AI 助手系统，集成多个 MCP 工具服务器，支持飞书和 Web 双平台接入，为团队提供知识库问答、日程管理、数据查询等智能化办公能力。

核心职责：

- 基于 Nanobot 的 Channel 适配器模式开发飞书和 Web 双平台接入层，实现消息格式的统一转换和路由，支持文本、图片、文件等多种消息类型
- 开发 5 个自定义 MCP Server：数据库查询工具（支持自然语言转 SQL）、文档检索工具（基于向量相似度）、日程管理工具（对接飞书日历 API）、Jira 集成工具（查询和创建 Issue）、通知工具（飞书群消息推送）
- 利用 Nanobot 的 Skills 渐进披露机制，设计 8 个领域 Skill（数据分析、周报生成、会议纪要整理等），实现工具的按需加载，system prompt token 占用减少 40%
- 实现基于 Redis 的语义缓存层，对相似查询命中率达 35%，LLM API 月度成本降低约 25%

项目成果：

- 系统上线后日均处理 500+ 次对话请求，用户满意度评分 4.2/5.0
 - 工具调用成功率 94%，平均响应时间 3.2 秒
 - 团队信息检索效率提升约 60%，减少了大量重复性问答沟通
-

模板 C：MCP 工具开发项目（适合偏协议/基础设施方向）

项目名称：企业级 MCP 工具服务器集群开发

项目周期：2026.03 - 2026.04

项目角色：核心开发者

技术栈：Python, MCP SDK, JSON-RPC 2.0, FastAPI, Docker, PostgreSQL, Elasticsearch

项目描述：

基于 MCP (Model Context Protocol) 协议规范，为企业内部 AI Agent 系统开发标准化的工具服务器集群，实现企业数据和业务系统与 AI 应用的标准化连接。

核心职责：

- 基于 MCP Python SDK 开发 6 个工具服务器，覆盖数据库查询（MySQL/PostgreSQL）、全文检索（Elasticsearch）、项目管理（Jira）、即时通讯（飞书/钉钉）、

文件系统操作、HTTP API 代理等能力

- 设计 MCP Server 的统一安全框架：实现 API Key 认证、输入参数 JSON Schema 验证、敏感操作确认机制（destructive annotation）、操作审计日志
- 实现 MCP Server 的 stdio 和 HTTP+SSE 双传输模式支持，本地开发使用 stdio 模式提升调试效率，生产环境使用 HTTP+SSE 模式支持远程访问和多用户共享
- 编写完整的工具描述文档，确保 LLM 能准确理解每个工具的用途和参数要求，将工具调用准确率从 78% 提升至 93%

项目成果：

- 6 个 MCP Server 已集成到 3 个不同的 AI Agent 应用中（Nanobot、Cursor、内部 Agent 平台），验证了 MCP 的「一次开发，多处复用」核心价值
- 将新 AI 应用接入内部系统的开发周期从平均 2 周缩短到 2 天
- 项目文档和设计方案被团队采纳为 MCP Server 开发规范

四、智能运维助手项目简历模板

项目名称：基于 Nanobot 的智能运维助手系统

项目周期：2026.02 - 2026.04

项目角色：后端开发 / 核心架构

技术栈： Python, HKUDS/nanobot, MCP 协议, Claude API, Prometheus, Grafana, Docker, Redis, PostgreSQL

项目描述：

基于 Nanobot Agent 框架构建智能运维助手系统，集成多个 MCP 工具服务器，支持日志分析、告警处理、服务器状态查询、故障自动诊断等运维场景，显著降低运维人员的工作负荷。

核心职责：

- 基于 Nanobot 的 AgentLoop 和 ReAct 循环，实现运维场景的多步推理——Agent 能自动判断告警类型、查询相关日志、定位根因并给出修复建议
- 开发 6 个运维领域 MCP Server：日志检索工具（对接 Elasticsearch）、服务器监控工具（对接 Prometheus API）、告警管理工具（对接 PagerDuty）、部署工具（对接 Kubernetes API）、数据库巡检工具、网络诊断工具
- 利用 Nanobot 的 MemoryConsolidator 记忆系统，实现运维知识的持久化——Agent 自动记忆历史故障模式和解决方案，相似故障再次发生时可快速匹配历史经验
- 设计运维 Skill 体系：按照故障类型（CPU 过高、内存泄漏、磁盘满、服务不可用等）定义专用 Skill，利用渐进披露机制按需加载

项目成果：

- 系统上线后覆盖 80% 的常见运维场景，平均故障诊断时间从 30 分钟缩短到 5 分钟
- 工具调用成功率 92%，运维知识库积累 500+ 条历史故障经验
- 夜间值班告警的自动处理率达到 45%，显著减轻了运维人员的工作压力

五、多平台客服 Bot 项目简历模板

项目名称：基于 Nanobot 的多平台智能客服系统

项目周期：2026.02 - 2026.04

项目角色：全栈开发 / 技术负责人

技术栈： Python, HKUDS/nanobot, MCP 协议, Claude API, FastAPI, Redis, PostgreSQL, 飞书/钉钉/微信开放平台

项目描述：

基于 Nanobot Agent 框架构建统一的多平台智能客服系统，通过 Channel 适配器

模式同时接入飞书、钉钉、企业微信和 Web 四个平台，为企业提供一体化的 AI 客服解决方案。

核心职责：

- 基于 Nanobot 的 Channel 适配器架构，开发四套平台适配器——每个适配器独立处理平台特有的消息格式、认证机制和回调验证，核心 Agent 逻辑完全解耦，新增平台只需开发适配器层
- 实现智能路由和会话管理：根据用户意图自动分配到售前咨询、售后支持、技术支持等不同 Skill 域，利用 Redis 实现跨平台的用户画像统一
- 开发 8 个客服领域 MCP Server：订单查询、物流追踪、退换货处理、FAQ 知识库检索、工单创建、优惠券查询、用户信息查询、人工坐席转接
- 利用 Nanobot 的并发控制机制（session_locks + _concurrency_gate），实现高并发场景下的稳定服务，支持 200+ 并发会话

项目成果：

- 四平台同时上线，日均处理 2000+ 次客服对话，用户满意度 4.3/5.0
 - 客服人力成本降低 35%，80% 的常见问题由 AI 自动解决
 - 新平台接入开发周期从 2 周缩短到 2 天，验证了 Channel 适配器模式的可扩展性
-

六、自定义 MCP Server 开发项目简历模板

项目名称：企业级自定义 MCP Server 工具集开发

项目周期：2026.03 - 2026.04

项目角色：核心开发者

技术栈： Python, MCP SDK, JSON-RPC 2.0, FastAPI, Docker, PostgreSQL, Elasticsearch, 飞书/钉钉 API

项目描述：

基于 MCP (Model Context Protocol) 协议规范，为团队的 Nanobot Agent 系统开发一套标准化的 MCP 工具服务器集群，实现企业内部各业务系统与 AI Agent 的标准化连接，践行「一次开发，多处复用」的 MCP 核心理念。

核心职责：

- 基于 MCP Python SDK 开发 8 个工具服务器，覆盖数据库查询 (MySQL/PostgreSQL)、全文检索 (Elasticsearch)、项目管理 (Jira/飞书项目)、即时通讯 (飞书/钉钉)、文件系统操作、HTTP API 代理、日程管理、审批流查询等能力
- 设计 MCP Server 的统一安全框架：实现 API Key 认证、输入参数 JSON Schema 验证、敏感操作确认机制 (destructive annotation)、操作审计日志
- 实现 MCP Server 的 stdio 和 HTTP+SSE 双传输模式支持，本地开发使用 stdio 模式提升调试效率，生产环境使用 HTTP+SSE 模式支持远程访问和多用户共享
- 精心设计每个工具的描述文档 (description 和 inputSchema)，通过迭代优化将 LLM 的工具调用准确率从 78% 提升至 95%

项目成果：

- 8 个 MCP Server 已集成到 3 个不同的 AI Agent 应用中 (Nanobot、Cursor、内部 Agent 平台)，验证了 MCP 的跨应用复用价值
 - 将新 AI 应用接入内部系统的开发周期从平均 2 周缩短到 2 天
 - 项目文档和设计方案被团队采纳为 MCP Server 开发规范
 - 开源 2 个通用 MCP Server 到社区，获得 200+ stars
-

七、其他可写项目方向

7.1 RAG 知识库问答系统

项目名称：企业知识库智能问答系统

技术栈： Python, LangChain, Milvus, BGE Embedding, Cohere Rerank, FastAPI, PostgreSQL, Docker

核心亮点：

- 实现混合检索（向量 + BM25），Rerank 精排后 NDCG@10 提升 12%
- 设计文档权限隔离方案，基于 RBAC 的元数据过滤
- 语义缓存命中率 30%，LLM API 月度成本降低 20%

7.2 多 Agent 协作系统

项目名称：基于多 Agent 的自动化代码审查系统

技术栈： Python, Nanobot SubagentManager, GitHub API, MCP, AsyncIO, Redis

核心亮点：

- 设计 Orchestrator-Workers 架构：主 Agent 分配审查任务，3 个专业 Worker Agent 分别负责代码风格、安全漏洞、性能问题的检查
- 实现 Agent 间的结构化通信协议，控制通信 token 冗余在 15% 以内
- 系统日均处理 50+ PR 审查请求，发现有效问题率 72%

7.3 AI 对话机器人（飞书/钉钉）

项目名称：飞书智能助手机器人

技术栈： Python, Nanobot, 飞书开放平台, MCP, Redis, Nginx

核心亮点：

- 基于 Nanobot Channel 适配器模式开发飞书接入层
- 集成 6 个 MCP 工具（日程查询、文档搜索、审批流等）
- 支持多用户并发（_concurrency_gate 控制），日均服务 200+ 用户

7.4 Prompt 工程优化平台

项目名称：Prompt 版本管理与 A/B 测试平台

技术栈： Python, FastAPI, PostgreSQL, Redis, React, Docker

核心亮点：

- 实现 Prompt 的版本管理、diff 对比、回滚能力
- 支持线上 A/B 测试，按流量比例分配不同版本的 Prompt
- 集成自动化评测（GPT-4 评分 + 人工抽样），评测覆盖率 100%

八、自我评价模板

模板 1：Agent 开发方向（应届/1-3 年）

- 对 AI Agent 系统有深入理解，系统研读过 Nanobot 框架源码（~4000行），掌握 AgentLoop、记忆管理、工具注册等核心模块的设计原理
- 熟悉 MCP 协议规范，有自主开发 MCP Server 的实践经验
- 具备扎实的 Python 编程基础，熟练使用 asyncio 异步编程
- 理解大模型原理（Transformer/Attention/RoPE），有模型训练和微调的实践经历
- 热爱开源，有技术博客写作习惯，善于学习和总结新技术

模板 2: LLM 应用方向 (2-5 年)

- 3 年 Python 后端开发经验, 1 年+ AI/LLM 应用开发经验
- 深入理解 Agent 架构设计, 有从 0 到 1 搭建企业级 AI 助手系统的经验
- 精通 RAG 系统设计和优化 (混合检索、Rerank、Chunk 策略), 有向量数据库 (Milvus) 的生产使用经验
- 掌握 Prompt 工程最佳实践, 有成本优化经验 (Prompt Cache、语义缓存、模型分级路由), 累计为团队节省 API 成本 30%+
- 具备系统设计能力, 注重代码质量和工程规范

模板 3: 算法方向 (硕博)

- 计算机科学硕士, 研究方向为自然语言处理/大模型
- 深入理解 Transformer 架构及其变体 (GQA、MoE、Flash Attention)
- 有完整的大模型训练经验: Pretrain → SFT → LoRA → DPO 全流程
- 熟悉 PyTorch 分布式训练 (DDP/FSDP/DeepSpeed), 有多 GPU 训练经验
- 发表/在投 NLP/LLM 相关论文 X 篇, 参与开源项目 X 个
- 了解 Agent 系统架构, 关注 AI Agent 前沿动态

九、完整简历示例

张三 | AI Agent 开发工程师

☎ 138-xxxx-xxxx | ✉ zhangsan@email.com | 🌐 github.com/zhangsan
📍 北京 | 期望薪资: 面议

教育背景

XX 大学 计算机科学与技术 本科 2022.09 - 2026.06

- GPA: 3.6/4.0 | CET-6: 550+
- 核心课程: 数据结构、操作系统、计算机网络、机器学习、深度学习

技术技能

- 编程语言: Python (精通)、TypeScript (熟悉)、Go (了解)
- AI/LLM: Prompt Engineering、RAG 系统、Agent 架构、MCP 协议
- Agent 框架: Nanobot (源码级理解)、LangChain/LangGraph
- 模型训练: PyTorch、Transformer 架构、SFT/DPO、LoRA 微调
- 后端开发: FastAPI、PostgreSQL、Redis、Docker、Nginx
- 工程工具: Git、CI/CD (GitHub Actions)、Linux

项目经历

基于 Nanobot 的企业智能助手系统 2026.02 - 2026.04

技术栈: Python, Nanobot, MCP, Claude API, FastAPI, Redis, Docker

- 系统研读 Nanobot 核心源码 (~4000 行), 深入理解 AgentLoop 的 ReAct 执行流程、MemoryConsolidator 的双层记忆架构、ToolRegistry 的注册表模式等核心设计
- 基于 MCP 协议开发 5 个自定义工具服务器 (数据库查询、文档检索、

日程管理、Jira 集成、飞书通知），实现了企业内部系统与 AI Agent 的标准化连接

- 开发飞书和 Web 双平台接入层，基于 Nanobot 的 Channel 适配器模式实现消息格式统一转换，支持文本和文件等多种消息类型
- 设计 8 个领域 Skill 并利用渐进披露机制实现按需加载，system prompt 的 token 占用减少 40%；实现 Redis 语义缓存，LLM API 成本降低 25%
- 系统上线后日均处理 500+ 次对话，工具调用成功率 94%，用户满意度 4.2/5

企业级 MCP 工具服务器集群开发 2026.03 - 2026.04
技术栈：Python, MCP SDK, JSON-RPC 2.0, FastAPI, Docker, PostgreSQL

- 基于 MCP 协议开发 8 个工具服务器，覆盖数据库查询、全文检索、项目管理、即时通讯等能力，实现企业内部系统与 AI Agent 的标准化连接
- 设计统一安全框架（API Key 认证、JSON Schema 验证、destructive 注解、审计日志），支持 stdio 和 HTTP+SSE 双传输模式
- 精心优化工具描述文档，将 LLM 工具调用准确率从 78% 提升至 95%

个人亮点

- 对 AI Agent 技术有浓厚兴趣，系统性学习并输出 12 篇深度技术文档
- GitHub 上有开源项目和技术博客，持续关注 Agent/MCP 前沿动态
- 良好的学习能力和文档写作习惯，善于从源码中理解设计思想

十、简历注意事项

10.1 必须避免的错误

错误类型	错误示例	正确做法
空洞描述	“负责项目的开发工作”	用 STAR 法则具体描述
缺少量化	“提升了系统性能”	“响应时间从 5s 降低到 2s”
技术堆砌	列出 20 个技术但无深度	精选 8-10 个有实践经验的
过度夸大	“精通 xxx”（实际只用过一次）	用「了解/熟悉/精通」分级
格式混乱	字体大小不一、对齐混乱	统一格式、简洁清爽
信息遗漏	没有联系方式或 GitHub	确保基本信息完整
篇幅过长	简历超过 2 页	应届生 1 页，社招 1-2 页
敏感信息	附上身份证号/家庭住址	只提供必要的联系信息

10.2 项目经历的描述规范

每个项目应包含：1. 项目名称 + 时间段 2. 技术栈列表（具体版本可省略）3. 3-5 条核心职责描述（用动词开头：设计、实现、优化、研究）4. 1-2 条量化成果

动词使用建议：- 高级感动词：主导、设计、架构、研究、优化 - 中级动词：实现、开发、集成、搭建、完成 - 避免使用：参与、协助、帮助（显得没有主导权）

10.3 技能熟练度分级标准

级别	含义	示例
精通	能讲出底层原理，有大量实践经验	Python（5 年 +，读过 CPython 源码）
熟悉	有项目经验，能独立完成开发任务	FastAPI（2 个项目，含中间件开发）
了解	学习过原理，有简单使用经验	Go（完成过一个小项目）

级别	含义	示例
----	----	----

重要提醒：「精通」是一个很重的词，面试官会围绕你标注为「精通」的技术深入追问。如果不确定，宁可写「熟悉」也不要写「精通」。

10.4 针对 ATS（简历筛选系统）的优化

大厂通常使用 ATS 自动筛选简历，优化策略：

1. 使用标准格式：避免花哨的排版、图标、图片，使用纯文本友好的格式
2. 关键词密度：确保 JD 中的核心关键词在简历中出现 2-3 次（自然融入）
3. 标准标题：使用常见的章节标题（教育背景、项目经历、技术技能）
4. **PDF** 格式：投递时使用 PDF 格式，避免 Word 格式在不同系统上显示错乱
5. 文件命名：姓名-岗位名-联系方式.pdf（如 张三-AI Agent 开发工程师-138xxxx.pdf）

十一、不同岗位的简历侧重点

11.1 AI Agent 开发工程师

重点突出：- Agent 框架源码理解（Nanobot AgentLoop、MemoryConsolidator 等）- MCP 协议实践（开发过 MCP Server）- 工具调用可靠性设计（超时、重试、截断）- 系统设计能力（高可用、并发控制）

项目顺序：Nanobot 项目放第一个

技能调整：将 MCP、Agent 框架、asyncio 放在技能列表前面

11.2 LLM 应用工程师

重点突出：- RAG 系统设计和优化经验 - Prompt 工程实践 - 成本优化经验（缓存、Prompt Cache）- API 集成和工程化能力

项目顺序：RAG 或 AI 助手应用项目放第一个

技能调整：将 RAG、向量数据库、Prompt Engineering 放在技能列表前面

11.3 大模型算法工程师

重点突出：- 完整的模型训练经验（Pretrain/SFT/DPO）- 架构理解（Transformer 变体、GQA、MoE）- 对齐技术（DPO vs PPO 对比实验）- 论文阅读和研究能力

项目顺序：模型训练相关项目放第一个

技能调整：将 PyTorch、Transformer、SFT/DPO/LoRA 放在技能列表前面

11.4 AI 产品经理

重点突出：- 对 Agent 能力和限制的理解 - 用户体验设计思考 - 数据分析和效果评估 - 竞品分析和行业认知

项目顺序：偏应用和用户侧的项目放第一个

技能调整：弱化技术细节，强调产品思维和业务理解

11.5 岗位简历对照表

简历要素	Agent 开发	LLM 应用	算法	产品
Nanobot 源码				
MCP 协议				
RAG 系统				
模型训练				
系统设计				
Prompt 工程				
工程实践				

十二、简历投递策略

12.1 投递前检查清单

- ☐ 简历中的技术关键词与目标 JD 匹配
- ☐ 每段项目经历都有量化成果
- ☐ 联系方式完整正确（手机号、邮箱、GitHub）
- ☐ 没有错别字和语法错误
- ☐ PDF 格式，文件名规范
- ☐ 技能熟练度分级准确，不会被追问露馅
- ☐ 项目时间线合理，没有不可解释的空窗期

12.2 精准投递策略

1. 每份 **JD** 微调简历：不要用同一份简历投所有岗位，至少调整技能顺序和项目侧重点
2. 投递配文案：在 Boss 直聘等平台打招呼时，用 2-3 句话说明你与岗位的匹配点
3. 内推优先：通过脉脉、即刻、微信群联系目标公司员工，内推通过率远高于海投
4. 时间选择：工作日上午 9-11 点投递效果最好，HR 在线概率最高

12.3 打招呼话术模板

您好！我是 XX 大学计算机专业应届生，对 AI Agent 开发方向有深入学习和实践：

1. 系统研读过 Nanobot 框架源码（4000行），理解 AgentLoop 等核心机制
2. 有 MCP Server 开发经验，开发过 5 个自定义工具
3. 有多平台 AI 客服系统的开发经验，支持飞书/钉钉/微信/Web 四平台接入

非常期待能加入贵团队！我的简历已附上，方便的话希望能聊一聊。

总结

简历是面试的第一道门槛，核心要做到：

1. 精准匹配：根据目标岗位调整关键词和项目侧重
2. 量化说话：用数字证明你的价值，避免空洞描述
3. 深度取胜：展示源码级理解、协议理解等深度能力
4. 格式专业：简洁、清晰、一页为佳
5. 持续更新：随着项目进展和学习深入持续完善简历

最重要的原则：简历中写的每一条都要做好被面试官深挖的准备。宁可少写两条也不要写自己答不上来的内容。

上一章：第 14 章 就业市场分析 下一章：第 16 章 STAR 面试话术